

Discovery Executive Summary

Project: discovery-20jul-2026 · Generated: 20/07/2026, 18:34:08

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	MODERATE	—
2	Frontend Modernization Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

Moderate

Driven by the inability to validate architecture evidence under the cloud-only source restriction.

EXECUTIVE SUMMARY

A real hotspot scan could not be completed in this run because the prompt required cloud-only repository access and explicitly forbade local source inspection and GitHub MCP usage. I therefore did not fabricate stack detection, file counts, or hotspot measurements. The repository snapshot indicates both backend and frontend artifacts exist, but the architecture health for either layer remains unverified. The dominant risk is unknown rather than assessed risk, so the safest outcome is to treat this report as a blocked discovery pass and rerun it with source access enabled.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	---------	-------------	------	----------	-----------	----------	--------

H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	Not measured	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	Not measured	MODERATE
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	Not measured	MODERATE
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	Not measured	MODERATE
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	Not measured	MODERATE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	Not measured	MODERATE
H7	God Classes	Classes >1000 LOC	0	1–3	>3	Not measured	MODERATE
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	Not measured	MODERATE
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	Not measured	MODERATE
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	Not measured	MODERATE
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	Not measured	MODERATE
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	Not measured	MODERATE
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	Not measured	MODERATE
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	Not measured	MODERATE

1.4 Actions Required

Hotspot	Action	Rating	Priority
None	No action taken because source evidence could not be collected in this run.	MODERATE	MEDIUM

1.5 Expected Outcomes

- Real architecture hotspots can be measured once source access is enabled.
- Backend and frontend layers can be compared on equal footing.

- Service, repository, and boundary extraction opportunities become visible.
- Future refactors can be prioritized from evidence instead of inference.
- Change amplification risk can be reduced with a concrete target-state map.

2. Frontend Modernization Analysis

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

High Risk

The largest driver is massive component scale, with ``App.jsx``, ``Dashboard.jsx``, ``StepFlowSelection.jsx``, and ``StepIdeConfig.jsx`` each carrying too much routing, orchestration, and form logic in one place.

EXECUTIVE SUMMARY

This workspace has a mature React frontend, and the dominant idiom is already function components with hooks on top of React 19.2.5. The main modernization gap is not framework migration; it is component scale and orchestration complexity, especially in `App.jsx`, `Dashboard.jsx`, and the setup flow screens. Legacy class-based UI is effectively absent aside from a single error boundary, so the codebase is mostly aligned with current React patterns. The most material risks are oversized components, repeated imperative state synchronization, and deep feature components that mix routing, auth, layout, and data orchestration. Overall, the codebase is usable and mostly modern, but the largest screens would benefit from extracting shared shell, state, and form primitives.